



Henderson Amateur Radio Club

Henderson Amateur Radio Club, Inc.

Sunday Night Net

Current System Description

Prepared:

April 2, 2024

Prepared by:

Tim Collier – K7JTC



Table of Contents

Objective	4
Description.....	4
General	4
Check-in List	5
Callout Roster	5
Net Content	6
Content Availability.....	6
System Context.....	6
Events.....	6
Sunday Night Net Administrator:	7
Net Control Team:	7
Amateur Radio Operator (both members and non-members):	7
Use Cases.....	7
Actors.....	7
Use Case Description	7
Create Weekly Script.....	8
Collect Announcement Information.	8
Create Callout Roster.....	8
Remove Duplicate Information:	8
Create Roster Normal Order:.....	9
Create Roster Reverse Order:	9
Create Roster Random Order:	9
Create Roster Suffix Order:	9
Clean up callout information:.....	10
Check Person's name:.....	10
Check Membership Status	10
Add New Non-Member (extends "Check Membership"):	10
Add New Member (extends "Check Membership"):.....	10
Make a member a non-member: (extends "Check Membership"):	10
Make a non-member a member: (extends "Check Membership"):	11
Schedule Net Control Operators.....	11
Conduct Sunday Night Net.....	11
Record check-ins to the Sunday Night Net.	11
Checks into the net.	12
Report Weekly Results.....	12
Information Requirements.....	12
HARC SNN QSO Information.....	13
Person Type Code	14
Person	14
Ham Club Online Membership List.....	15



Call Out Cutoff Days	17
System Specification	17
Alternatives	17
Spreadsheet.....	17
Database.....	18
APPENDIX I – Context Diagram	19
Appendix II – Use Case Diagram.....	20
Appendix III – Data Model	21
Appendix IV – Physical Database Design - Tables.....	22
Appendix V – Physical Database Design - Views	23
Appendix VI – Database Terminology.....	24
SQLite:.....	24
Table:.....	24
View:.....	24
Relationship:	25
Appendix VII – Data Definition Language	26
Tables.....	27
t_call_out_cutoff_days.....	27
t_hcol_export	28
t_net_qso.....	30
t_pers.....	31
t_pers_typ.....	32
Views (sometimes referred to as Data Manipulation Language or DML)	33
v_call_out_cutoff_days	33
v_callout_roster_reverse	34
v_checkins_most_recent_net	35
v_count_qso_by_date	36
v_current_call_out_cutoff_days	37
v_harc_snn_callout_roster_random	38
v_harc_snn_callout_roster_suffix.....	39
v_harc_snn_callout_roster.....	40
v_hcol_export.....	42
v_maint_add_person_to_pers.....	43
v_maint_add_as_mem.....	44
v_maint_cha_nonmember_to_member	45
v_maint_latest_hcol_join_date.....	46
v_maint_remove_as_member.....	47
v_maint_show_dupes	48
v_net_qso	49
v_pers	50
v_potential_callout_roster_size.....	51



Objective

Henderson Amateur Radio Club's (HARC) Sunday Night Net has several related objectives:

- Provide a means by which the club can test the repeater network.
- Provide members a vehicle to test their UHF equipment.
- Train members in how to function as a net control operator.
- Provide a means by which members have “an excuse” to use their radios.
- Provide the club with information regarding members (if a member has not checked in for some time, an inquiry as to their well-being could be made).
- Provide an opportunity for non-members to see and hear about HARC's activities, thereby generating interest in joining the club.

Description

General

The Henderson Amateur Radio Club, Inc. sponsors a weekly check-in net each Sunday at 7pm local time.

It is a controlled net with a designated net control operator running the net.

The Sunday Night Net (SNN) content consists of about ten minutes of announcements (or QST's) followed by a roster call-out seeking check-ins. After the roster is called, the net control operator seeks check-ins from member stations who were not called, followed by guest station check-ins.

The following shows a rough outline (along with estimated elapsed times) of the script currently being used:

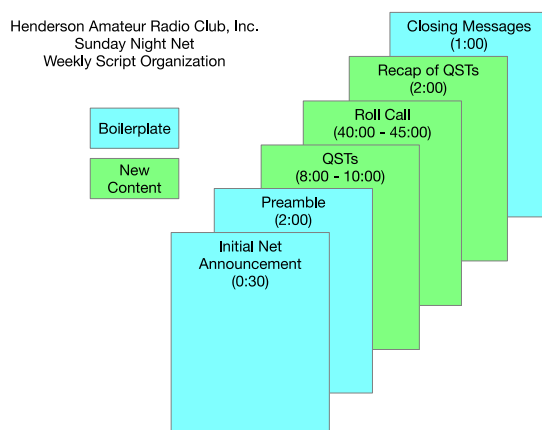


Figure 1 - Henderson Amateur Radio Club Sunday Night Net Script Structure



In the past, the SNN was immediately followed at 8pm by a very popular net called the Tek-Net. For that reason, the total running time of the SNN was kept to less than 60 minutes. The moderator of the Tek-Net (Earl WB6AMT) became a silent key (SK) last year, and nobody else has stepped forward to revive the Tek-Net. However, there is hope that someone will step forward. The club still wants the SNN kept to under an hour.

Check-in List

A list of those stations checking in is kept. As of February 26, 2024, there were over 14,000 check-ins, made up of almost 490 distinct callsigns contained in the list.

The check-in list may be updated in real time using a logging program. The list may also be updated at a later date during the week using the club's recordings. All activity on the HARC repeater network is recorded on [Broadcastify](#). The Broadcastify archives are kept for long enough to allow the person updating the list to access Sunday Night Nets for several weeks in the past.

Each week, the check-in list is queried looking for those callsigns who have a check-in date within the allowable window (e.g. have checked in one time during the past 28 days). Those meeting the selection criteria form the call-out roster.

Callout Roster

The callout roster is generated weekly.

The callout roster is generally in Member / non-member, last name (A-Z), first name (A-Z) order.

The first net of the month is in a different order. So far, the different order has been one of:

- random (the most common order),
- by suffix
- reverse order (member / non-member, last name (Z to A), first name (Z to A)).

We have found that, to keep the net running time reasonable, a max of 100 callouts should occur. Currently, the roster contains between 80 and 90 callsigns. The way the list has been limited, has been by creating a window within which a station must have checked in. Currently, the window is set at 28 days; if the last time a station checked into the net was 28 days ago or less, their callsign will appear on the callout roster.

There have been times when there have been more than 100 stations who checked-in within the 28 day window. For those instances, the window is reduced. Most of the time, if the 28 day checkout calls for more than 100 callsigns to appear on the list, 21 days will reduce that number to below 100.



Net Content

Net content is prepared by the Net Control Administrator. A script is prepared each week containing some general club information (boiler plate) along with specific announcements (QST's) of interest to the general amateur radio community (e.g. meeting dates, club events, other items of interest).

The content has historically been in the following order:

- Welcome to the SNN
- General information about the club, the net, and the net control operator.
- An invitation for emergency or priority traffic to break into the net at any time.
- Announcements of interest to the amateur radio community (QST's).
- Roll Call (both from the check-in list and call for other stations to check-in).
- Re-cap of the announcements in date order (earliest first).
- Closing comments.

The QST's, roll call roster, and recap constitute the majority of work each week. The remaining items remain static.

The net control operator can choose to read directly from the script (most do), but, should they wish to say things in their own words, that is always encouraged.

Content Availability

Once the content is created, it is made available to the designated net control operator for that week, and also to the rest of the net control team. Should something happen to the primary net control operator (e.g. radio failure), another net control operator can pick up the net and continue. It should be noted, however, that has not been necessary thus far.

System Context

(See [Appendix I – Context Diagram](#))

Events

The manual and automated processes that define the Sunday Night Net may be thought of as a business system.

The HARC Sunday Night Net System must interact with other similar business systems. The interactions are events that are generated either from the HARC Sunday Night Net System to the external entity or generated from the individual stakeholder systems to the HARC Sunday Night System. The events include:



Sunday Night Net Administrator:

- Sunday Night Net is Scheduled.
- Sunday Night Net is Concluded.

Net Control Team:

- Sunday Night Net is Scheduled

Scheduled Net Controller (a subset of Net Control Team):

- Conducts the Sunday Night Net.

Amateur Radio Operator (both members and non-members):

- Hears callsign called.
- Hears call for late or missed check-ins.

Use Cases

(See [Appendix II - Use Case Diagram](#))

An event will cause the corresponding system to react. The reactions are things the respective systems must do when the event occurs. These reactions constitute use cases.

A use case outlines, from an external entity's point of view, a system's behavior as it responds to an event.

Actors

Actors participating in the Sunday Night Net System include:

- *Sunday Night Net Administrator* - Responsible for oversight of the net.
- *Net Control Team* - Group of operators who take turns running the net. Every week, each Team member receives all the documents necessary to conduct the net as a backup resource.
- *Net Controller* - a member of the Net Control Team who runs a particular Sunday Night Net session.
- *Amateur Radio Operator* - An amateur radio operator, who is either a member or a non-member, who participates in the weekly Sunday Night Net.

Use Case Description



Create Weekly Script.

(Uses “Collect announcement information”)

The Sunday Night Net Administrator creates the weekly script. The weekly script is composed of some introductory boilerplate, an announcement section, a callout roster, a wrap-up summary section, and the concluding boilerplate.

Collect Announcement Information.

“Create Weekly Script” uses announcement information collected from club activities by the Sunday Night Net Administrator. Currently, this information is collected on an ad-hoc basis with no formal process in place. The current administrator uses the club’s calendar, information obtained from various meetings, as well as emails that have been received.

Create Callout Roster.

(Extended by “Create Roster Normal Order”, “Create Roster Reverse Order”, “Create Roster Random Order”, Create Roster Suffix Order”. “Remove Duplicate Information”)

The roster is the document created by the Sunday Night Administrator and given to the Sunday Night Net Controller. The callout roster contains the callsign, first name, last name, and membership status for each person who is in the callout window (e.g., checked in at least one time during the past 28 days). If the person has indicated they have a preferred name, then that name is used as the first name. Otherwise, the first name that appears on the license is used.

The HARC Board of Directors made the policy decision that, since the Sunday Night Net is for the members, the members should always be called before any non-members are called. Therefore, regardless of the order of the roster, all members will always be called before any non-members.

Information appearing on the current callout roster includes:

- Callsign
- First Name
- Last Name
- Member / non-member
- Comments (empty space for writing various comments).

Remove Duplicate Information:

This use case removes duplicate entries from the callout roster.

The selection criteria is, generally, looking for distinct occurrences of the combination of callsign, first name, and last name. There have been occasions that have occurred when the person has upgraded their license and, somehow, the first and last name are altered.

Since the callsign remains the same, but the names have been slightly altered, the selection criteria treats that as two separate instances. When the roster is printed out, the callsign will appear on the



list twice, with slightly different names. It is confusing to call the same callsign two times during the net.

With the name changes, the call sign may appear in different parts of the callout roster, making it difficult to pick up the duplication on the fly.

“Create Callout Roster” is extended by the following:

Create Roster Normal Order:

(Extends “Create Callout Roster”)

The names on the roster appear in the following order:

- Member/non-member (members are always called before non-members)
- Last name (A-Z)
- First name (or preferred name) (A-Z)
- Callsign

Create Roster Reverse Order:

(Extends “Create Callout Roster”)

The names on the roster appear in the following order:

- Member/non-member (members are always called before non-members)
- Last name (Z-A)
- First name (or preferred name) (Z-A)
- Callsign

Create Roster Random Order:

(Extends “Create Callout Roster”)

A random number is assigned to each person on the callout roster. The names on the roster appear in the following order:

- Member/non-member (members are always called before non-members)
- Random Number (not printed)

Create Roster Suffix Order:

(Extends “Create Callout Roster”)

The suffix of the callsign is extracted (the letters that appear after the district number). For example, for callsign K9ABC, the suffix is ‘ABC’. The names on the roster appear in the following order:

- Member/non-member (members are always called before non-members)
- Suffix (not printed)



- Last Name (A-Z)
- First Name (A-Z)

Clean up callout information:

The roster is dependent on having an accurate list of persons from which to choose. As time progresses, there are club members who leave the club, there are people who checked into the net as non-members who later join the club, there are new members who join the club and there is the chance the duplicates can find their way onto the callout roster.

Check Person's name:

Check the roster to be sure the correct first name is being used; the preferred name if available, otherwise the person's first name.

Check Membership Status

Over time, the person's membership status in HARC might change. There are several checks to be made.

The checks are made using the club's membership database of record. Currently, that is the member information saved in the HamClubOnline (HCOL) system. If a person appears on the HCOL roster as an active member, they are considered to be a HARC member. If they do not appear, they are not a member. There are several other lists being kept by various club officers, however, the Board of Directors decided that the HCOL roster would be treated as the official list.

Add New Non-Member (extends "Check Membership"):

If a person has checked into the net, and their callsign does not appear on the HCOL roster, add the person's information to the list of people who have checked into the net as a non-member.

Add New Member (extends "Check Membership"):

If a person has checked into the net, and their callsign appears on the HCOL roster, add the person's information to the list of people who have checked into the net as a member.

Make a member a non-member: (extends "Check Membership"):

Review the list of people who have checked into the net as a member. Compare the callsigns on that list with the HCOL roster. If the callsign does not appear on the HCOL list, change the status of the person from "member" to "non-member". This is important as the initial sort on the callout roster is by member / non-member with member coming first.



Make a non-member a member: (extends “Check Membership”):

Review the list of people on the HCOL list of members. Compare the callsigns on that list with the callsigns contained in the list of people who have checked into the net. If the callsign found on the HCOL list is found on the check-in list, and is listed as a non-member, change the status of the person from “non-member” to “member”. If the person is already classified as a member, do nothing. This is important as the initial sort on the callout roster is by member/non-member with members coming first on the callout roster.

Schedule Net Control Operators.

The Sunday Night Net Administrator sets the schedule for the upcoming time period (currently, quarterly). The Administrator will query the pool of net control operators seeking availability. Assignments are then made, with an attempt made to schedule evenly (no one net controller takes on more than their share).

Conduct Sunday Night Net

The net is started at 7pm Pacific Time each Sunday Evening with an announcement at 6:55pm Pacific that the net is about to begin.

The net control operator assigned that evening then reads the prepared script. Once the QSTs are complete, the roll call is made. After completing the roll call list, the net controller will then ask for late or missed check-ins, and visiting station check-ins. After roll call, a recap is read from the script and the net winds down with some boilerplate announcements.

Typically, the net control operator does not print out the script, but, rather, reads it from an electronic device. The roster, although included with the script, itself, is also sent as a separate document. This allows the net control operator to keep track of where in the roll call they are, and to make notes, such as someone who has additional traffic that might take longer than one minute.

Each check-in is acknowledged in some way depending on the net control operator; each has their own style. Acknowledgements are important to let the Amateur Radio Operator know they have been heard. Quick signal reports may also be part of the acknowledgement (“...We have you checked in, although your signal was a little scratchy ...”).

Record check-ins to the Sunday Night Net.

The Net Controller keeps track of who checks into the net. At some time, that information must be captured so that “Create callout roster” knows who has checked in within the current window (28 days, currently).

The Net Control Administrator has been listening to the nets over time and enters the check-in information in real time, for the most part. On those occasions when the Net Control Administrator cannot capture the check-ins as they happen, the Net Control Administrator will use the recordings made by the club on Broadcastify.



Checks into the net.

The Amateur Radio Operator listens for their callsign to be called and establishes communication with the Net Control Operator. At this time there are no required pieces of information which the operator must provide. However, the operators are asked during the net to identify which repeater they are using.

If the operator does not hear their callsign during the roster callout, time is allotted at the end of the net for late check-ins or visiting stations. When the call for late check-ins is issued, the net controller must deal with stations doubling with each other and must also listen closely for the callsigns. Most operators find this part of the net to be the most challenging, as well as the most fun.

Report Weekly Results

While the FCC has yet to request any information regarding the HARC Sunday Night Net, it is good practice to be ready with which stations checked in on which week, with the time, if available.

The Club is also interested in how many participated in the Sunday Night Net. Current participation, as of the end of February 2024, varies between 65 and 75 check-ins per week created by 90 call-outs and 10 late check-ins.

Information Requirements

(See [Appendix III - Data Model](#))

The information about persons, places, things, or concepts required to operate the Henderson Amateur Radio Club, Inc. Sunday Night Net is straightforward.



HARC SNN QSO Information

Information about individual contacts is necessary. The Net QSO Information is characterized by:

Attribute	System Name	Description
Unique Identifier	qso_key	A number that uniquely identifies this contact
Operator Callsign	call	The operator's amateur radio callsign
Net Identifier	net_id	A string of characters that identifies to which net this person checked-in. Currently, the net_id has the form HARC-SNN- ccyyymmdd, where
		- cc = century (20)
		- yy = year (24)
		- mm = two digit month
		- dd = two digit day
		A net held on July 4, 2024 would have the form HARC-SNN-20240704.
Contact's First Name	f_nam	First name of the contact. Currently, this field is being populated by information from the FCC database, which does not store preferred names.
Contacts Last Name	l_nam	Last name of the contact
Date the contact was made	qso_date	The date on which the contact was made
Time the contact was made	qso_time	The time at which the contact was made.



Person Type Code

Attribute	System Name	Description
Person Type Code	pers_typ_cd	A unique identifier of the person type code.
		Person Type Code keeps track of the various types of memberships. Currently the values are:
		- 0 = non-member.
		- 1 = member.
Person Type Description	pers_typ_desc	- 2 = member, but this is an old callsign.

Person

A place is needed to store a contact's preferred name. For example, the FCC might have the legal first name of "Francis", but the person prefers to go by "Frank". Or maybe they are legally a "William" but prefer "Bill", or some other preferred name. Also needed is the information about whether the person is a HARC member, or not.

A person is characterized by:

Attribute	System Name	Description
Person's callsign	callsign	the person's callsign.
Person's First Name	f_nam	the person's first name (legal name)
Person's Last Name	l_nam	The person's last name (legal name)
Person's Preferred Name	pref_nam	he name by which the person prefers to be called.
Person's HARC Membership Status	pers_typ_cd	a code indicating the HARC membership status of the person (member, non-member). The codes are associated with the Person Type Code



Ham Club Online Membership List

Ham Club Online (HCOL) contains the list of club members. By agreement among the HARC Board of Directors, the HCOL list is the “membership database of record”; if a person appears on the HCOL list, they are a member, if not, the person is a non-member.

The membership roster may be downloaded from HCOL in two different ways:

1. As a comma separated variable file (CSV) suitable for loading into most spreadsheet programs (MS Excel, MACOS Numbers, etc.).
2. As the output from an Application Program Interface (API) command.

The Ham Club Online Membership List of record is characterized by the following:

Attribute	System Name	Description
Member number	mem_num	a unique number used by the HCOL system.
Roster Status	roster_status	Active or inactive.
First Name	f_nam	First name (from FCC records)
Nickname	pref_nam	Preferred name or nickname.
Last Name	l_nam	Last name. (from FCC records)
Call sign	Call	Callsign (from FCC records)
License Class	lic_cls	License Class (Novice, Technician, General, Advanced, Amateur Extra, etc.) (from FCC records)
Date License expires	lic_exp_dat	Date license expires. (from FCC records)
License Number	lic_num	FCC Registration Number (or FRN). (from FCC records)
Licensing Agency	lic_agncy	usually the FCC
Street Address	street	Street number and name (from FCC records)
City	city	City name (from FCC records)
State	st	State , two character code. (from FCC records)
Postal Code	zip	Zip code (from FCC records)
Country	couontry_cd	Country code
Latitude	lat	latitude of the address.
Longitude	lon	longitude of the address
Elevation	elev	elevation of the address
HCOL e-mail address	hcol_email	Email address used by the HCOL system
HCOL e-mail address status	hcol_email_stat	Status of the HCOL email address (active or inactive0



Attribute	System Name	Description
Club e-mail address	club_email	Email address used by the club for its communications
Club e-mail address status	club_email_stat	Status of club email address.
Primary Phone Number	phone_pri	Primary telephone number
Primary Phone Number Type	phone_pri_typ	Type of Primary telephone Number
Primary Phone Number Publish	phone_pri_pub	Publish the Primary telephone number?
Alternative Phone Number	phone_alt	Alternative telephone number
Alternative Phone Number Type	phone_alt_typ	Alternative telephone number type
Alternative Phone Number Publish	phone_alt_pub	Publish the Alternative telephone number?
Emergency Contact	emer_cntct	Who to contact in case of an emergency. Used mainly for ARES deployments.
Date joined the club	join_dat	Date the person's membership became active.
Date left the club	left_dat	Date the person resigned from the club.
Club Rank	club_rank	Club Rank (voting, non-voting, officer).
Club Position	club_pos	Club position (President, Vice-President, Treasurer, Secretary, Director-at-Large (2 positions)).
Delegate Text	dele	Text field inserted at the request of another club. HARC has been using this field to store GMRS license numbers.
Membership Type	mem_ty	Membership Type (Regular, Family)
Membership Expiration Date	mem_exp_dat	Date the membership expires
Amount of money due to the club	club_bal	Amount of money the member owes the club. If the. Member joined the club, for example, but did not submit a payment, the balance-due will be reflected in this number.
Date of Birth	dob	Date of Birth
Private Member	priv_mem	Private Member
User Timezone	Tz	Timezone in which the HCOL and Cub operate.
Date last logged in	lst_login	Date the member last logged into the HCOL system
Number of different roles	role_cnt	Describes how many roles the member has assigned to them.



Call Out Cutoff Days

The callout roster is based on those who have checked into the net within specified period of time. It was decided to track the time windows over time. The most recent entry is the current time window.

Attribute	System Name	Description
Date Established	date_established	Date the recent window is established
Number of Days	number_of_days	Number of days in the current window.

System Specification

Determining what technology to use to meet the requirements must consider both how to store the information and how to retrieve the information.

The solution must be at no, or very low, cost to the club.

The amount of data that needs to be saved and accessed is relatively small and not too dynamic by database standards. The weekly net generates around 60-70 check-ins each week. That would amount to 60-70 updates of the database, per week. That is not a lot of transactions.

There are only a few dependencies in the information which translates to a simple database structure.

Much of the information that must be retrieved can be displayed by simple row and column output.

Alternatives Spreadsheet

The original net was managed with several spreadsheets (MS Excel). The row and column nature of a spreadsheet was conducive to tracking the QSOs. The small number of data elements were reasonably manageable by creating different spreadsheets inside of a workbook, and then linking the relevant data items together.

However, as the number of callouts and net participation increased, the spreadsheets became cumbersome.



Database

The system was migrated to a [SQLite database](#). The decision to migrate was based on:

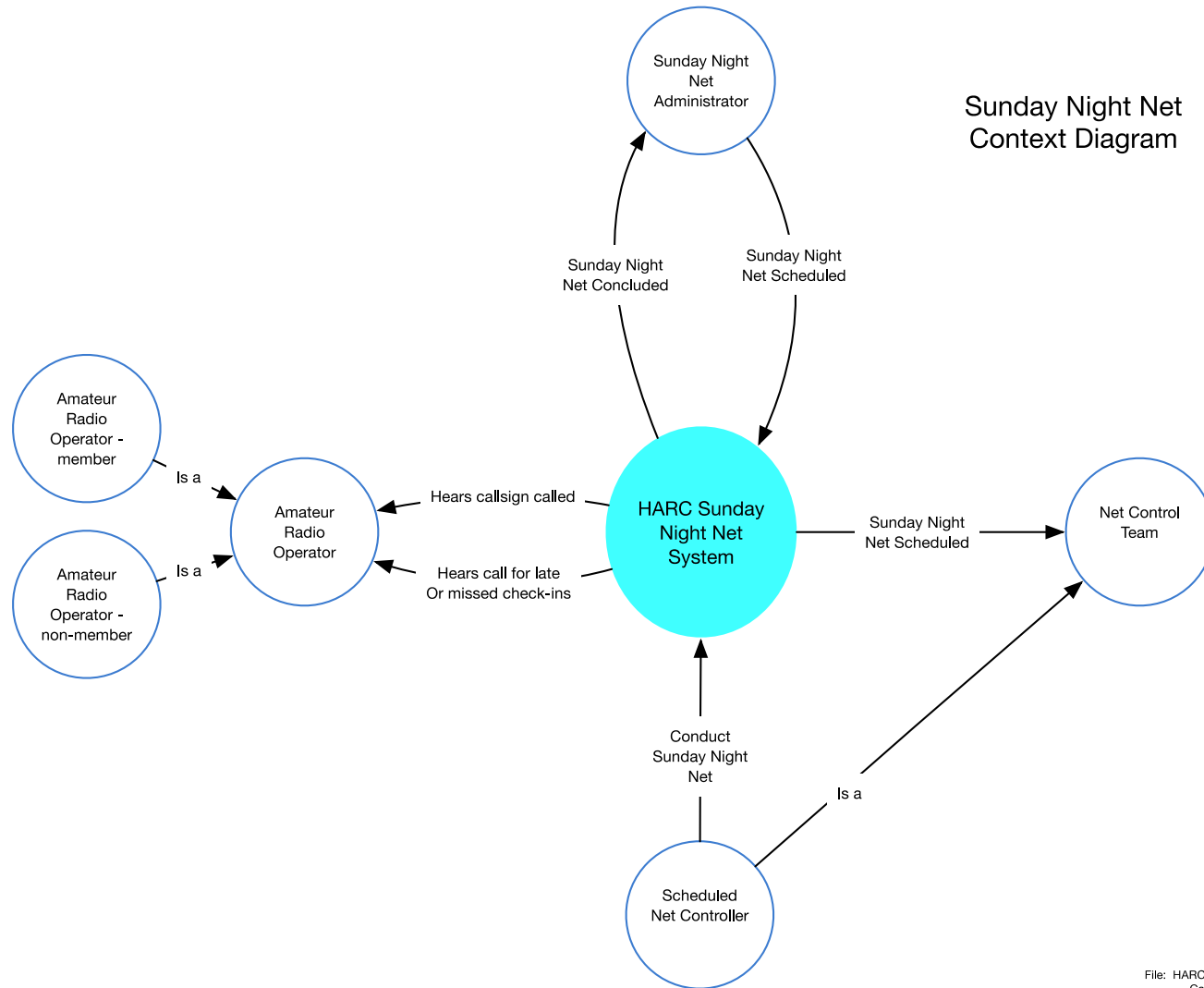
- SQLite is already installed on many MacOS, Windows, and Linux computers as part of their operating system and would not require additional purchases. If not already installed, the installation of SQLite is straightforward.
- SQLite is a well known database product, supporting many operating system features. (To see if SQLite is already installed, at the command line prompt type *Sqlite3*)
- The data definition language (DDL) and data manipulation language (DML) is Structured Query Language (SQL), a well-established computer language.
- Browsing and Editing the SQLite database can be done with an open-source desktop application called [SQLiteStudio](#). SQLiteStudio is available for the Mac, Windows, or Linux operating systems. Open Source means anyone is free to use it for any purpose (GPL(v3) License), SQLiteStudio has been in development and general use since early 2007.
- SQLite can import data from a variety of sources using CSV formats.

Please refer to [Appendix VI – Database Terminology](#) for a further explanation of database terminology.

Also, please refer to [Appendix IV – Physical Database Design – Tables](#) and [Appendix V – Physical Database Design – Views](#).

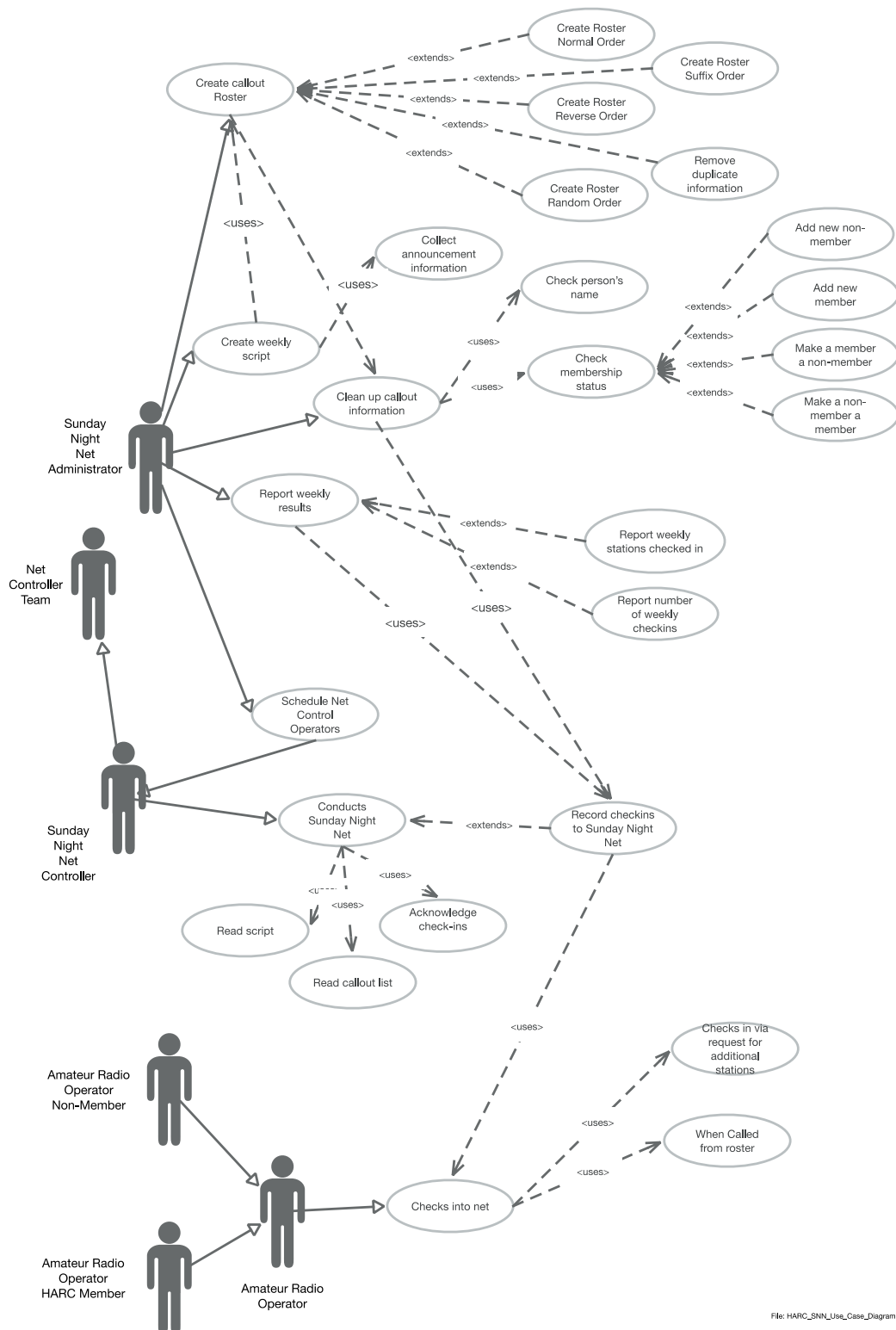


APPENDIX I – Context Diagram



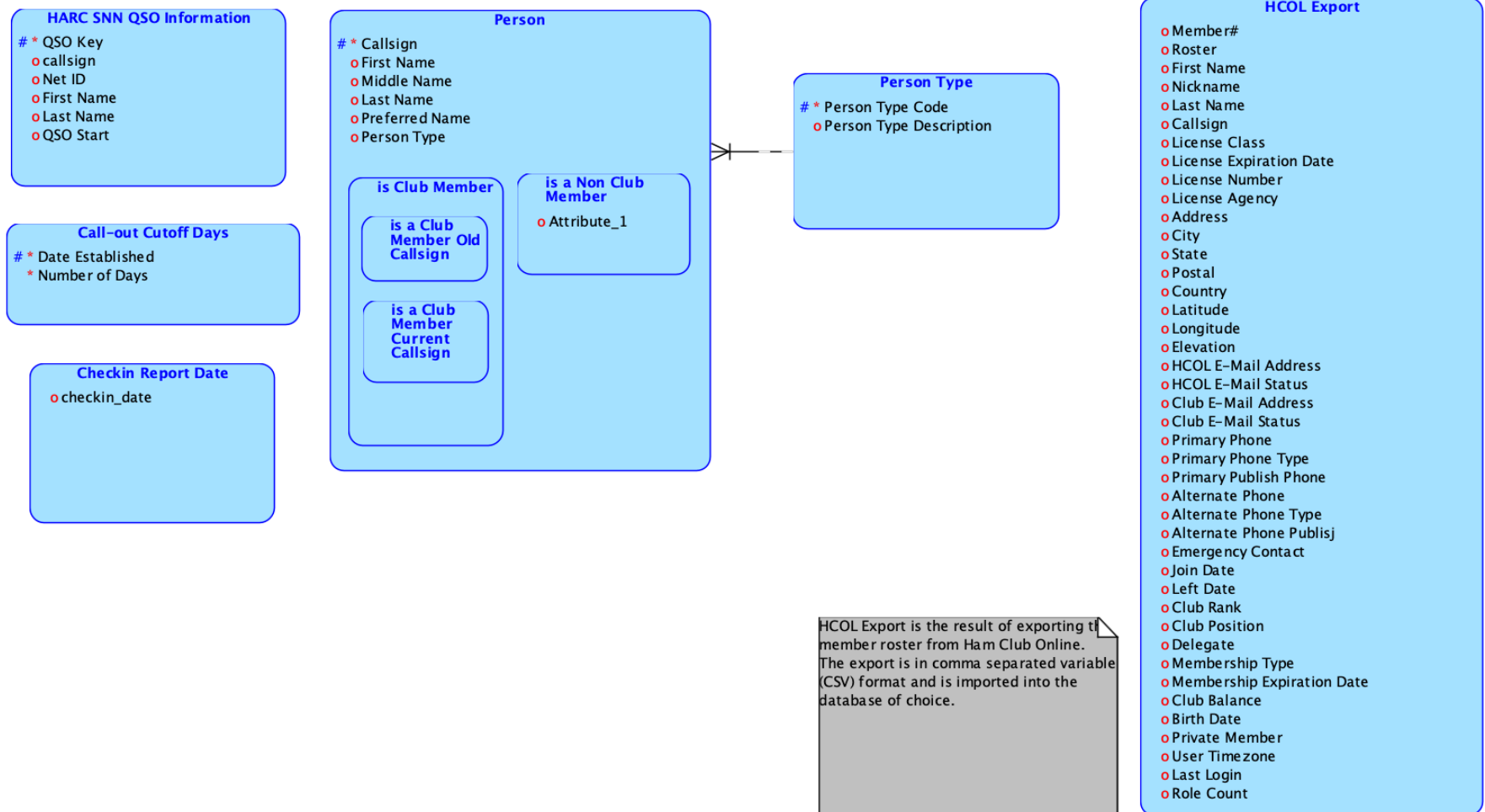


Appendix II – Use Case Diagram



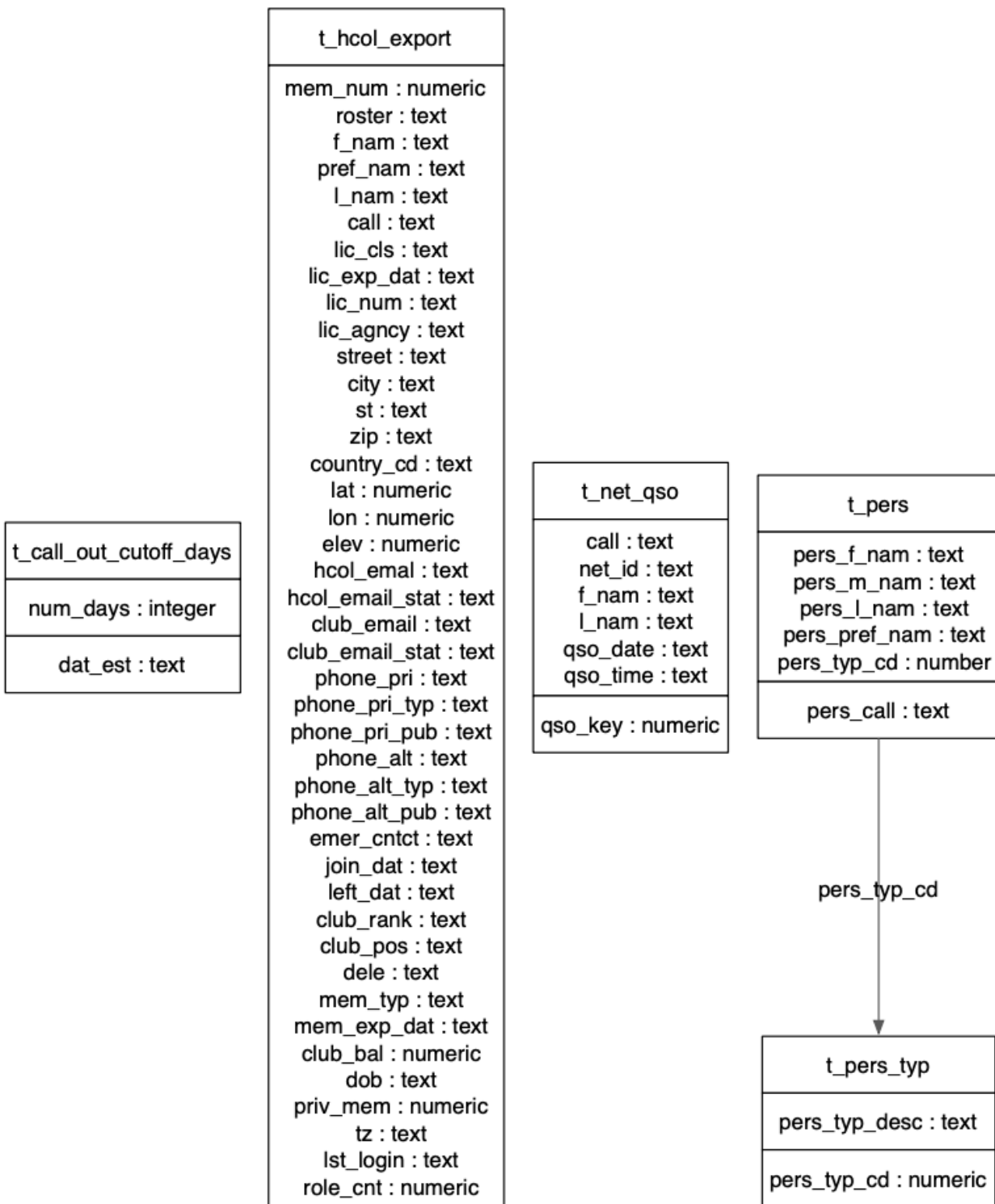


Appendix III – Data Model



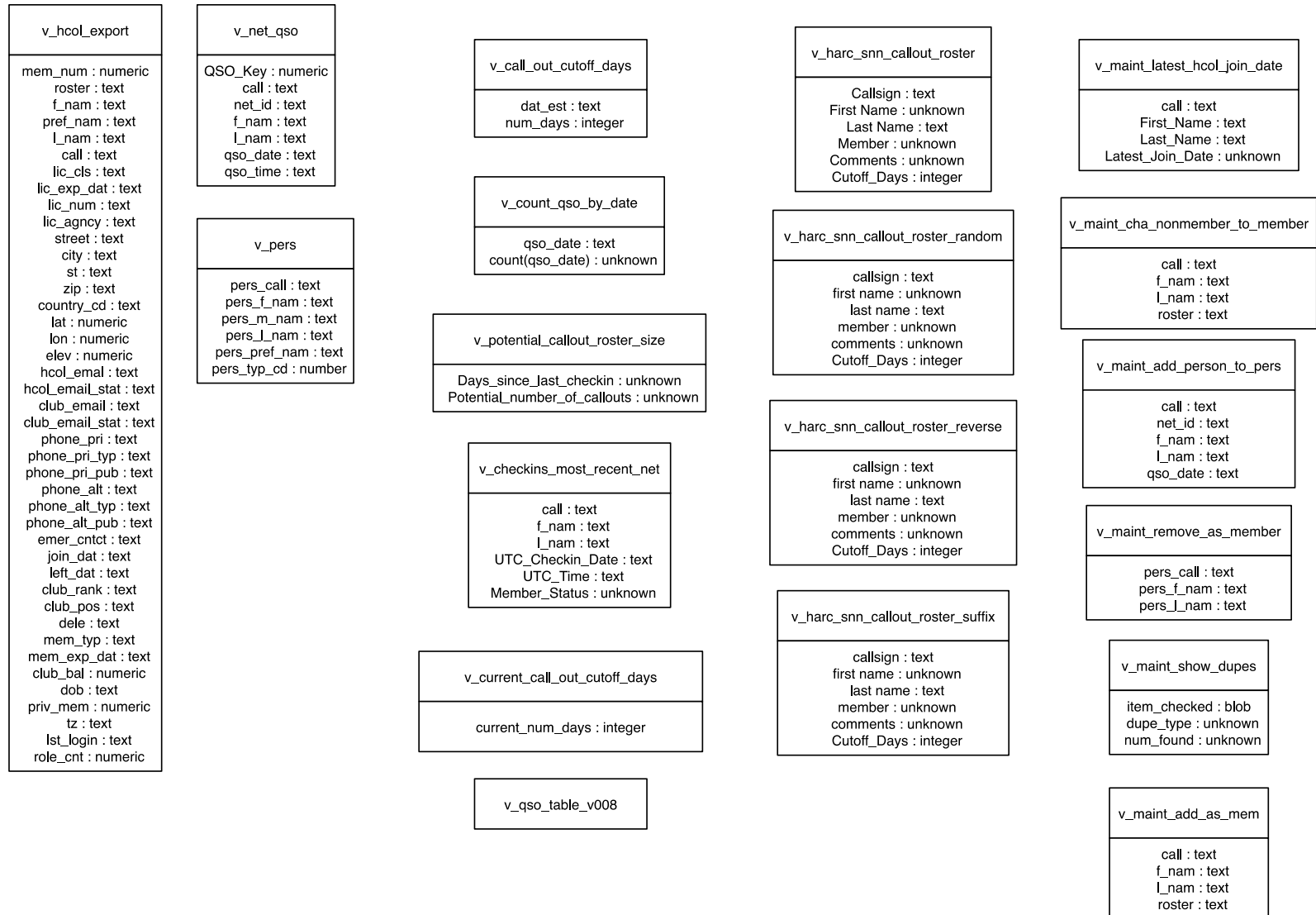


Appendix IV – Physical Database Design - Tables





Appendix V – Physical Database Design - Views





Appendix VI – Database Terminology

SQLite:

SQLite is a lightweight, self-contained, and serverless relational database management system (RDBMS) that is widely used for embedded database applications and small to medium-sized projects. Developed by D. Richard Hipp, SQLite is designed to be simple, efficient, and easy to integrate into various applications without the need for a separate database server.

One distinctive feature of SQLite is its self-contained nature, as it operates as a single ordinary disk file for the entire database, making it easy to deploy and manage. This file-based approach eliminates the need for a separate database server process and simplifies setup and administration tasks.

Despite its lightweight footprint, SQLite supports most of the SQL92 standard and provides features typical of relational databases, including tables with typed columns, indexes, triggers, and views. It is ACID-compliant (Atomicity, Consistency, Isolation, Durability), ensuring data integrity and reliability, even in the event of system failures.

SQLite is particularly well-suited for scenarios where simplicity, low resource requirements, and ease of deployment are crucial, such as in embedded systems, mobile applications, and standalone desktop applications. Its popularity also extends to web development, where it can be employed for small to medium-scale projects, prototyping, and testing. The seamless integration of SQLite with various programming languages, including C, Python, and Java, further contributes to its widespread adoption across diverse domains.

(See also [<https://www.sqlite.org/>])

Table:

A table is a fundamental structure in a relational database that organizes and stores data in a tabular format. It consists of rows and columns, where each row represents a record or a data entity, and each column represents a specific attribute or field of that entity.

Tables are designed to follow a specific schema, which defines the data types and constraints for each column. The combination of rows and columns allows for efficient storage, retrieval, and manipulation of data.

View:

A view is a virtual table that is based on the result of a SELECT query. Unlike a physical table, a view does not store the data itself; instead, it provides a dynamic, up-to-date representation of the data stored in one or more underlying tables.

Views are useful for simplifying complex queries, abstracting underlying table structures, and presenting a customized perspective on the data to users. They can also be used to restrict access to certain columns or rows, providing a layer of security.



Relationship:

In the context of relational databases, a relationship refers to the association between tables based on common columns. These relationships are established to connect related data across different tables, promoting data integrity and avoiding data redundancy.

There are different main types of relationships:

One-to-One (1:1): Each record in the first table corresponds to exactly one record in the second table, and vice versa.

One-to-Many (1:N): Each record in the first table can be associated with multiple records in the second table, but each record in the second table is associated with only one record in the first table.

Many-to-Many (M:N): Multiple records in the first table can be associated with multiple records in the second table, and vice versa. This type of relationship is typically implemented using an intermediary table, known as a junction or associative table.

Understanding these concepts is crucial for designing efficient and normalized relational databases, ensuring data accuracy, and facilitating complex queries.



Appendix VII – Data Definition Language

There is a naming convention to help keep the data objects separate. Tables begin with “t_” while views begin with “v_”.

Tables are the place where the data are stored. Data that resides in a table may be created, read, edited, or deleted. All data updates occur in tables.

Views, on the other hand, are “virtual” tables, which are, generally, the result of a Structured Query Language (SQL) query. Since the data that is seen in the views originates in tables, the data shown by the Views cannot be modified. Data modification takes place only in the tables.

Views are created “on-the-fly” as needed. Most of the “work” in producing output from the data comes from the SQL executed to create the views.

It is a common practice to create a View for every Table and then have all data retrieval activities make use of the View, not the Table. The benefit of using this design structure is, should a change to a Table become necessary (e.g., split the table, or add columns to the table, etc.), all programs accessing that information in the Table will not have to be opened for maintenance. Only the underlying View would have to be modified to redefine the View and all subsequent queries accessing the View could be left unchanged.



Tables

t_call_out_cutoff_days

```
/*
Table: t_call_out_cutoff_days
-----
Author:  Tim Collier
Purpose: Store information about the number of days in the cutoff window,
along with the date the number of days became effective.
*/
create table main.t_call_out_cutoff_days
(
    dat_est  TEXT      not null primary key,
    num_days INTEGER not null
)
without rowid;
```



t_hcol_export

```
/*
Table:  t_hcol_export
-----
Author:  Tim Collier
Purpose:  Store the results of the comma separated variable (CSV) file
downloaded from the HanClubOnline (HCOL) system.  HCOL was designated as
the membership database of record.  This table will have to be updated
occasionally as this is the table from which the maintenance views
determine whether to add a new member, or change an existing members
information in t_pers.
*/
create table main.t_hcol_export
(
    mem_num          NUMERIC,
    roster           TEXT,
    f_nam            TEXT,
    pref_nam         TEXT,
    l_nam            TEXT,
    call             TEXT,
    lic_cls          TEXT,
    lic_exp_dat      TEXT,
    lic_num          TEXT,
    lic_agncy        TEXT,
    street           TEXT,
    city             TEXT,
    st              TEXT,
    zip              TEXT,
    country_cd       TEXT,
    lat              NUMERIC,
    lon              NUMERIC,
    elev            NUMERIC,
    hcol_email       TEXT,
    hcol_email_stat  TEXT,
    club_email       TEXT,
    club_email_stat  TEXT,
    phone_pri        TEXT,
    phone_pri_typ    TEXT,
    phone_pri_pub    TEXT,
    phone_alt        TEXT,
    phone_alt_typ    TEXT,
    phone_alt_pub    TEXT,
    emer_cntct       TEXT,
    join_dat         TEXT,
    left_dat         TEXT,
    club_rank        TEXT,
    club_pos         TEXT,
    dele            TEXT,
    mem_typ          TEXT,
    mem_exp_dat      TEXT,
    club_bal         NUMERIC,
    dob             TEXT,
```



```
priv_mem      NUMERIC,  
tz            TEXT,  
lst_login     TEXT,  
role_cnt      NUMERIC  
);
```



t_net_qso

```
/*
Table: t_net_qso
-----
Author:  Tim Collier
Purpose: Store information about individual QSOs.
Note:   The system uses the view v_net_qso for most of its processing
needs.  If v_net_qso can be defined from other sources, this table is not
necessary.  The logging system currently being used (MacLoggerDX) stores
its logging information in a Sqlite database in a table called
qso_table_v008.  The view v_net_qso can be specified using qso_table_v008,
which means this table (t_net_qso) would not be necessary.
*/
create table main.t_net_qso
(
    qso_key    Numeric not null    primary key,
    call       TEXT,
    net_id     TEXT,
    f_nam      TEXT,
    l_nam      TEXT,
    qso_start  REAL,
    qso_date   TEXT,
    qso_time   TEXT
);
```



t_pers

```
/*
Table:  t_pers
-----
Author:  Tim Collier
Purpose:  Keep track of the people who check into the net.  Also, keep
track of the name by which the person prefers to be called (e.g. Bill
instead of William).  Their membership status is also captured.

The column pers_typ_cd contains a number (0,1, 2, or 3 at the present
time) that points back to the t_pers_typ table to pick up the words that
go along with 0,1,2,3 (non-member, member,
                    member-old call, silent key).
*/
create table main.t_pers
(
    pers_call      TEXT    not null  primary key,
    pers_f_nam     TEXT,
    pers_m_nam     TEXT,
    pers_l_nam     TEXT,
    pers_pref_nam  TEXT,
    pers_typ_cd    NUMBER not null
constraint pers_typ_FK references main.t_pers_typ
);
```



t_pers_typ

```
/*
Table: t_pers_typ
-----
Author:  Tim Collier
Purpose: Provide a place to store the textual names that go along with
the number codes stored in other tables, such as t_pers (pers_typ_cd).
*/
create table main.t_pers_typ
(
    pers_typ_cd    NUMERIC not null primary key,
    pers_typ_desc  TEXT      not null
);
```




Views (sometimes referred to as Data Manipulation Language or DML)

v_call_out_cutoff_days

```
/*v_call_out_cutoff_days
-----
Replicates current table t_call_out_cutoff_days.
```

Author: Tim Collier

Purpose: Callsigns will appear on the callout roster if they have checked in at least once during The cutoff period. Table t_call_out_cutoff_days keeps a records for how long the period is. Most likely, the cutoff time will be 28 days (4 weeks).

```
*/
CREATE VIEW v_call_out_cutoff_days(dat_est, num_days) AS
SELECT dat_est
       , num_days
FROM t_call_out_cutoff_days;
```



v_callout_roster_reverse

```
/*  
v_harc_callout_roster_reverse  
-----  
Author:  Tim Collier
```

Purpose: Lists the order of v_harc_snn_callout_roster (the usual Sunday Night Net roster) in reverse order.

```
*/  
CREATE VIEW v_harc_snn_callout_roster_reverse AS  
SELECT callsign,  
       "first name",  
       "last name",  
       member,  
       comments,  
       Cutoff_Days  
FROM (SELECT callsign,  
            "first name",  
            "last name",  
            member,  
            comments,  
            Cutoff_Days  
      FROM (SELECT *  
            FROM v_harc_snn_callout_roster)  
      ORDER BY 4,  
            "last name" DESC,  
            "first name" DESC);
```



v_checkins_most_recent_net

```
/*  
q_checkin_list  
-----
```

Author: Tim Collier
Date: April 10, 2023

Purpose: Produce a list of the actual check-ins for the most recent Sunday Night Net.

```
*/
```

```
CREATE VIEW v_checkins_most_recent_net AS
```

```
select call  
      , f_nam  
      , l_nam  
      , qso_date as UTC_Checkin_Date  
      , qso_time as UTC_Time  
      , case b.pers_typ_cd  
          when 1  
            then 'Member'  
          else 'Non-Member'  
      end      Member_Status  
  
from v_net_qso a  
     join v_pers b on a.call = b.pers_call  
where a.qso_date >= (select max(qso_date) from v_net_qso);
```



v_count_qso_by_date

```
/*  
v_count_qso_by_date  
-----  
Author:  Tim Collier
```

Purpose: Count the number of QSOs by date.

```
*/  
CREATE VIEW v_count_qso_by_date AS  
select qso_date  
      , count(qso_date)  
from v_net_qso  
group by qso_date  
order by 1;
```



v_current_call_out_cutoff_days

```
/*  
v_current_call_out_cutoff_days  
-----  
Author:  Tim Collier
```

Purpose: Gives a quick view of what the current callout period (cutoff days) is.
*/

```
CREATE VIEW v_current_call_out_cutoff_days AS  
SELECT num_days AS current_num_days  
FROM v_call_out_cutoff_days  
WHERE dat_est = (SELECT max(dat_est)  
                  FROM v_call_out_cutoff_days);
```



v_harc_snn_callout_roster_random

```
/*  
v_harc_snn_callout_roster_random  
-----  
Author:  Tim Collier
```

Purpose: Lists the contents of v_harc_snn_callout_roster (the usual callout roster) in random order.
Output order still differentiates between member and non-member, but is random after that.

```
*/  
CREATE VIEW v_harc_snn_callout_roster_random AS  
SELECT callsign,  
       "first name",  
       "last name",  
       member,  
       comments,  
       Cutoff_Days  
FROM (SELECT random(),  
            callsign,  
            "first name",  
            "last name",  
            member,  
            comments,  
            Cutoff_Days  
      FROM (SELECT *  
            FROM v_harc_snn_callout_roster)  
      ORDER BY 5,  
              1);
```



v_harc_snn_callout_roster_suffix

```
/*  
v_harc_snn_callout_roster_suffix  
-----  
Author:  Tim Collier
```

Purpose: Display the v_harc_snn_callout_roster in order of suffix (the letters that appear after the zone number).

```
*/  
CREATE VIEW v_harc_snn_callout_roster_suffix AS  
select callsign  
      , "first name"  
      , "last name"  
      , member  
      , comments  
      , Cutoff_Days  
from (select callsign  
      , case cast(substr(callsign, 2, 1) as integer)  
          when 0 then substr(callsign, 4, length(callsign))  
          else substr(callsign, 3, length(callsign))  
      end as "Suffix"  
      , "first name"  
      , "last name"  
      , member  
      , comments  
      , Cutoff_Days  
from (select * from v_harc_snn_callout_roster)  
order by 5, 2);
```



v_harc_snn_callout_roster

```

CREATE VIEW v_harc_snn_callout_roster AS SELECT DISTINCT a.call AS Callsign,
      CASE WHEN v_pers.pers_call IS NULL THEN a.f_nam
      ELSE ifnull(v_pers.pers_pref_nam, v_pers.pers_f_nam)
      END AS [First Name],
      v_pers.pers_l_nam AS [Last Name],
      CASE WHEN v_pers.pers_typ_cd = '1' THEN 'member'
      ELSE 'non-member'
      END AS ember,
      ' AS
Comments,
      (
      SELECT current_num_days
      FROM v_current_call_out_cutoff_days
      )
      AS Cutoff_Days
FROM v_net_qso a
      LEFT JOIN
      v_pers ON trim(upper(a.Call) ) = trim(upper(v_pers.pers_call) )
/*
      Select those records that show a contest id beginning with HARC-SNN (Sunday Night Net)
      * as well as those that are within the required number of days (probably 28 days).
      * There may be some special cases where a call sign is always included */
      * Selection Criteria
      -----
      pers_typ - 0 or 1 (non-member or member, excludes duplicate callsign informatoin
      contest_id must contain 'HARC' (usually it will be HARC-SNN)
      qso_date must be within the call_out_cutoff_days (v_current_call_out_cutoff_days)
      callsign (call) does not appear in the Silent Key List
      callsign (call) does appear in the mandatory call list
      */
WHERE (pers_typ_cd = 0 OR
      pers_typ_cd = 1) AND
      substr(a.net_id, 1, 4) = 'HARC' AND
      a.qso_date > (
      SELECT date('now', '-' ||
      (

```




```
SELECT current_num_days
FROM v_current_call_out_cutoff_days
)
||' days')
)
AND
a.call NOT IN (
    'WB6MIE'/* Tony Dinkel */
    , 'WB6AMT'/* Earl Lizardi */
    , 'W7DVU'/* Doug Smith */
)/* Silent Key List, comma separated */
OR
a.call IN (
    'N6YFN' /* Lawrence Roney Net Control Operator */
    , 'K7JTC' /* Tim Collier Net Control Operator */
    , 'KF7JAF' /* Dave Le Blanc Net Control Operator */
    , 'KD7YMO' /* Gerry Grabo Net Control Operator */
    , 'KF4KF' /* Karol Frodsham Net Control Operator */
    , 'KG7XD' /* Gene McAvoy Net Control Operator */
    , 'N7WDZ' /* Gene Heiter New Call Sign. */
    , 'K7ZEX' /* Tony Fetcho Net Control Operator */
)-- insert comma-separated call signs that will always appear in call-out list
ORDER BY 4,
        3,
        2;
```



v_hcol_export

```
/*  
  v_hcol_export  
  -----  
  Author:   Tim Collier  
  
  Purpose:  Base view of table t_hcol_export  
  */  
CREATE VIEW v_hcol_export AS  
select *  
from t_hcol_export;
```



v_maint_add_person_to_pers

```
/*
v_maint_add_person_to_pers
-----
Author:  Tim Collier
Purpose: Identify those persons who have checked into the net as persons who need to be
        added to the table t_pers.
*/
CREATE VIEW v_maint_add_person_to_pers AS select
    call
    ,net_id
    ,f_nam
    ,l_nam
    ,qso_date
from v_net_qso
where call not in
    (select pers_call from v_pers);
```



v_maint_add_as_mem

/*

v_maint_add_as_mem

Author: Tim Collier

Purpose: Part of the weekly maintenance.

Lists those members who appear in the t_hcol_export table (a copy of the HamClubOnline Member Export table) but do not appear as members in the t_pers table.

(The view does not change any values. See query q_maint_insert_new_member to automatically insert these members).

*/

CREATE VIEW v_maint_add_as_mem AS

select call

, f_nam

, l_nam

, roster

from v_hcol_export

where (roster = 'Active' or roster = 'Expiring Soon')

and length(call) > 0

and call not in

(select pers_call

from v_pers

where pers_typ_cd = 1);



v_maint_cha_nonmember_to_member

/*

v_maint_cha_nonmember_to_member

Author: Tim Collier

Purpose: Part of the weekly system maintenance.

List those that appear in the t_pers table who are identified as non-members, but who do appear on the t_hcol_export table (the Ham Club Online Membership database of record), and are, therefore, members.

(This will not change the person's status. See query q_maint_update_non-mem_to_mem.sql which will make the necessary changes).

*/

CREATE VIEW v_maint_cha_nonmember_to_member AS

select call

, f_nam

, l_nam

, roster

from v_hcol_export

where (roster = 'Active' or roster = 'Expiring Soon')

and call in

(select pers_call from v_pers where pers_typ_cd = 0);



v_maint_latest_hcol_join_date

```
/*  
v_maint_latest_hcol_join_date  
-----  
Author:  Tim Collier
```

Purpose: List the most recent date in the t_hcol_export table. If members have joined since that date, it would be wise to download the HCOL Members Export file, and reload the t_hcol_export table so the system is dealing with the most recent information.

```
*/
```

```
CREATE VIEW v_maint_latest_hcol_join_date AS  
select call  
      , f_nam          as "First_Name"  
      , l_nam          as "Last_Name"  
      , max(join_dat) as Latest_Join_Date  
from v_hcol_export;
```



v_maint_remove_as_member

```
/*  
v_maint_remove_as_member  
-----  
Author:  Tim Collier
```

Purpose: Part of the weekly maintenance of the system.
Identify those in the t_pers_table who are listed as members, but who do not appear in the membership database of record (t_hcol_export) and are, therefore, not members. (This does not change the values in the t_pers table. See query q_maint_update_member_to_nonmember.sql)

```
*/  
CREATE VIEW v_maint_remove_as_member AS  
select pers_call  
      , pers_f_name  
      , pers_l_name  
from v_pers  
where pers_typ_cd = 1  
      and pers_call not in  
      (select call  
       from v_hcol_export  
       where (roster = 'Active' or roster = 'Expiring Soon'));
```



v_maint_show_dupes

```
/*  
v_show_dupes  
-----  
Author: Tim Collier
```

Purpose: Duplicate callouts (calling the same person more than once) sometimes find their way into the roster. One example was a member who changed their mailing address. Their last name was changed to Mr. xxx. That was a name that was not present in the t_pers table, so it was added. That created two entries in the callout roster, both with the same call sign and first name, but slightly different last name.

This query counts the number of times a callsign appears and the number of times an operator's full name appears on the roster. It lists the output by number of instances found, so multiple call signs, or multiple names, will appear at the top of the list.

If no duplicates exist, the count of the call sign or name will be 1.

```
*/  
CREATE VIEW v_maint_show_dupes AS  
select callsign          as item_checked  
      , 'Callsign'       as dupe_type  
      , count(callsign)  as num_found  
from (select callsign, "First Name", "Last Name" from v_harc_snn_callout_roster)  
group by callsign  
  
Union  
  
select full_name         as item_checked  
      , 'Name'          as dupe_type  
      , count(full_name) as num_found  
from (select callsign, "First Name", "Last Name", "Last Name" || ", " || "First Name" as full_name  
      from v_harc_snn_callout_roster)  
group by full_name  
order by 3 DESC, 1;
```




v_net_qso

```
/*  
v_net_qso  
-----
```

Author: Tim Collier

Purpose: Base view for most other views.

Based on the table t_net_qso, but could be created from any table that contains the proper columns: call, net_id, f_nam, l_nam, qso_date, qso_time.

For example, v_net_qso has been created from the qso_table_v008 table, which is created from a MACOS-based logging program called MacLoggerDX. However, v_net_qso could be created from another table, as well.

For clarity's sake, v_net_qso is created from t_net_qso, which is created from qso_table_v008.

```
*/  
CREATE VIEW v_net_qso  
    (QSO_Key, call, net_id, f_nam, l_nam, qso_date, qso_time)  
AS  
SELECT QSO_Key  
    , call  
    , net_id  
    , f_nam  
    , l_nam  
    , qso_date  
    , qso_time  
FROM t_net_qso;
```



v_pers

/*

v_pers

Author: Tim Collier

Purpose: Store call information about anyone who has ever checked into the Sunday Night Net; that is currently over 600 individual call signs. When generating the callout roster, the system looks to this table to pick up call sign and name information, specifically, preferred name information.

It was created because, while we can pick up people's names from the FCC database, their preferred names may be different.

When the net first started up, several soon-to-be-members were bothered by the fact that the club could not remember their preferred name.

Each entry has a person type code denoting to what category the person belongs; member, non-member, of member with an old call sign.

*/

```
CREATE VIEW v_pers (pers_call, pers_f_nam, pers_m_nam, pers_l_nam, pers_pref_nam, pers_typ_cd) AS
SELECT pers_call
      , pers_f_nam
      , pers_m_nam
      , pers_l_nam
      , pers_pref_nam
      , pers_typ_cd
FROM t_pers;
```



v_potential_callout_roster_size

```
/*  
v_potential_call_out_roster_size  
-----
```

Author: Tim Collier

Purpose: When the net was first established, the BOD asked for about 10 minutes of announcements (QSTs) and also requested the net be done by 8pm (1 hour). The 8pm time was due to the Tech Net coming on line.

After much experimentation we found that 10 minutes of QSTs and 100 or less callouts fit the requirements. The size of the callout list is controlled by the callout period that is used; there will be more callouts if we use a 56 day (8 week) window, as opposed to a 28 day (4 week) window.

This view will show what the potential number of callouts

```
*/  
CREATE VIEW v_potential_callout_roster_size AS  
select 7 as Days_since_last_checkin  
 , count(call) as "Potential_number_of_callouts"  
from (select distinct call  
      from v_net_qso a  
      LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))  
      where (pers_typ_cd = 0 or pers_typ_cd = 1)  
      and substr(a.net_id, 1, 4) = 'HARC'  
      and qso_date > (select date('now', '-' || 7 || ' days'))  
      order by 1)  
  
union  
  
select 14 as Days_since_last_checkin  
 , count(call) as "Potential_number_of_callouts"  
from (select distinct call  
      from v_net_qso a  
      LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))  
      where (pers_typ_cd = 0 or pers_typ_cd = 1)  
      and substr(a.net_id, 1, 4) = 'HARC'
```



```
    and qso_date > (select date('now', '-' || 14 || ' days'))  
order by 1)
```

union

```
select 21          as Days_since_last_checkin  
    , count(call) as "Potential_number_of_callouts"  
from (select distinct call  
    from v_net_qso a  
        LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))  
    where (pers_typ_cd = 0 or pers_typ_cd = 1)  
        and substr(a.net_id, 1, 4) = 'HARC'  
        and qso_date > (select date('now', '-' || 21 || ' days'))  
    order by 1)
```

union

```
select 28          as Days_since_last_checkin  
    , count(call) as "Potential_number_of_callouts"  
from (select distinct call  
    from v_net_qso a  
        LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))  
    where (pers_typ_cd = 0 or pers_typ_cd = 1)  
        and substr(a.net_id, 1, 4) = 'HARC'  
        and qso_date > (select date('now', '-' || 28 || ' days'))  
    order by 1)
```

union

```
select 35          as Days_since_last_checkin  
    , count(call) as "Potential_number_of_callouts"  
from (select distinct call  
    from v_net_qso a  
        LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))  
    where (pers_typ_cd = 0 or pers_typ_cd = 1)  
        and substr(a.net_id, 1, 4) = 'HARC'  
        and qso_date > (select date('now', '-' || 35 || ' days'))  
    order by 1)
```



union

```
select 42          as Days_since_last_checkin
, count(call) as "Potential_number_of_callouts"
from (select distinct call
      from v_net_qso a
      LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))
      where (pers_typ_cd = 0 or pers_typ_cd = 1)
      and substr(a.net_id, 1, 4) = 'HARC'
      and qso_date > (select date('now', '-' || 42 || ' days'))
      order by 1)
```

union

```
select 49          as Days_since_last_checkin
, count(call) as "Potential_number_of_callouts"
from (select distinct call
      from v_net_qso a
      LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))
      where (pers_typ_cd = 0 or pers_typ_cd = 1)
      and substr(a.net_id, 1, 4) = 'HARC'
      and qso_date > (select date('now', '-' || 49 || ' days'))
      order by 1)
```

union

```
select 56          as Days_since_last_checkin
, count(call) as "Potential_number_of_callouts"
from (select distinct call
      from v_net_qso a
      LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))
      where (pers_typ_cd = 0 or pers_typ_cd = 1)
      and substr(a.net_id, 1, 4) = 'HARC'
      and qso_date > (select date('now', '-' || 56 || ' days'))
      order by 1)
```

union



```
select 63          as Days_since_last_checkin
, count(call) as "Potential_number_of_callouts"
from (select distinct call
      from v_net_qso a
      LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))
      where (pers_typ_cd = 0 or pers_typ_cd = 1)
            and substr(a.net_id, 1, 4) = 'HARC'
            and qso_date > (select date('now', '-' || 63 || ' days'))
      order by 1)

union

select 70          as Days_since_last_checkin
, count(call) as "Potential_number_of_callouts"
from (select distinct call
      from v_net_qso a
      LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))
      where (pers_typ_cd = 0 or pers_typ_cd = 1)
            and substr(a.net_id, 1, 4) = 'HARC'
            and qso_date > (select date('now', '-' || 70 || ' days'))
      order by 1)

union

select 'All Time' as Days_since_last_checkin
, count(call) as "Potential_number_of_callouts"
from (select distinct call
      from v_net_qso a
      LEFT JOIN v_pers on trim(upper(a.Call)) = trim(upper(v_pers.pers_call))
      where (pers_typ_cd = 0 or pers_typ_cd = 1)
            and substr(a.net_id, 1, 4) = 'HARC'
      -- and qso_date > (select date('now', '-' || 70 || ' days'))
      order by 1);
```